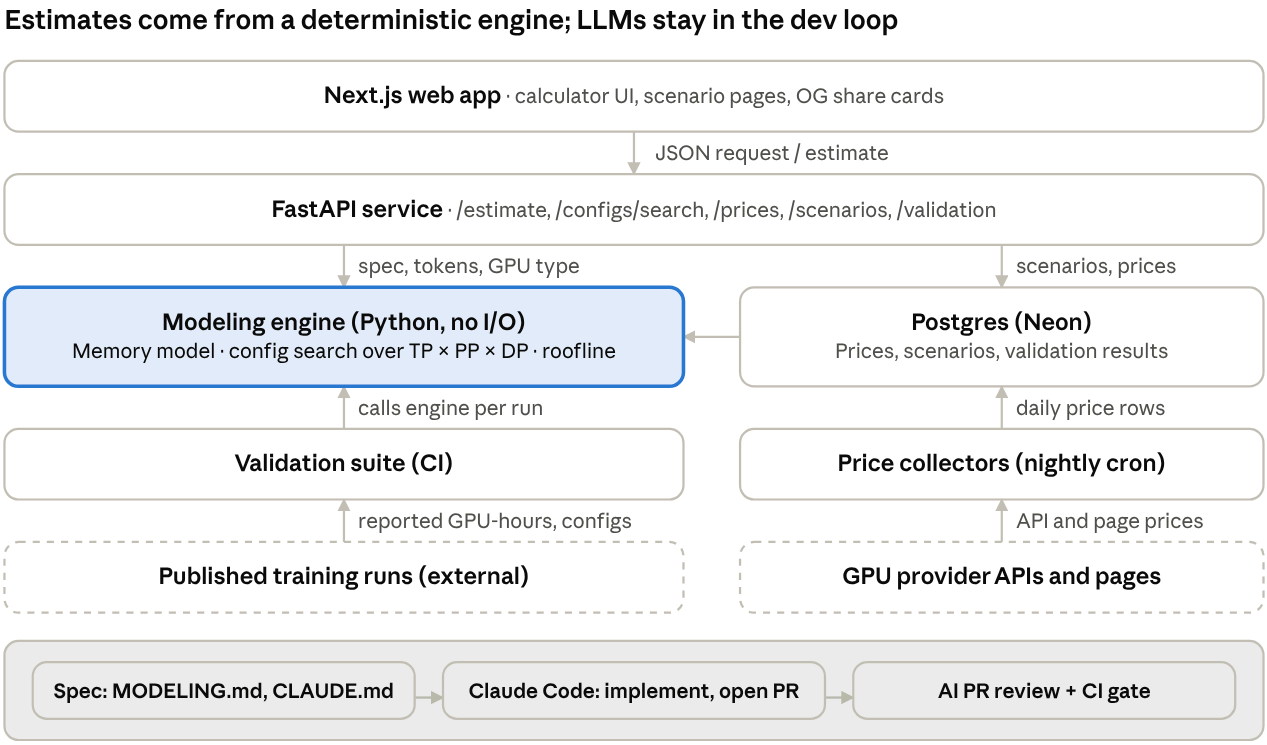
ComputeQuote: Technical Specification

Business Context Summary

ComputeQuote predicts what an AI training or inference workload will cost, from first principles and at current market prices. Teams plan $10K–$10M runs with "6 × parameters × tokens ÷ peak throughput" spreadsheets that ignore memory feasibility, achieved efficiency and fast-moving GPU prices, producing wrong budgets and mis-sized commitments. Users are ML engineers planning a run (primary), founders and finance leads budgeting compute, and analysts checking public cost claims; all decide before any GPU is rented, where a 2–3× error changes the decision.

Success metrics: GPU-hours within ±25% of reported figures for ≥5 published runs (DeepSeek-V3, Llama 3); cost exact to the cent and feasibility verdicts matching published configs; config search p95 under 1 s; ≥8 providers refreshed daily with stale entries flagged; new user to shareable estimate in under 2 minutes; ≥85% engine test coverage.

Architecture



ComputeQuote architecture · runtime path plus the LLM-assisted development loop

Requests flow down from the browser to the engine; prices and published runs flow up into it. The engine does no I/O, so it is tested without a database and rerun on published runs in CI. No runtime component calls an LLM; LLM tools act only in the development loop.

API Design

A stateless layer over the engine; every request and response is a Pydantic model, so the OpenAPI schema doubles as documentation for teammates and agents. Price collectors are plugins sharing one `fetch() -> list[PriceObservation]` interface over provider APIs (Vast.ai, RunPod) and pricing pages. No endpoint calls an LLM at runtime.

Endpoint	Input → output
`POST /v1/estimate/train`	Model spec, tokens, GPU type, precision → feasible config (TP/PP/DP, ZeRO), memory, step time, MFU, GPU-hours, cost

Endpoint	Input → output
`POST /v1/estimate/infer`	Model spec, GPU type, batch, lengths, utilization → throughput, max batch, $/M tokens, API break-even
`POST /v1/configs/search`	Model spec, GPU type and count → ranked feasible configs; infeasible ones with the reason
`GET /v1/prices`	GPU type, provider, date range → normalized $/GPU-hr history with staleness flags
`POST /v1/scenarios`, `GET /v1/scenarios/{id}`	Estimate request → short ID; GET replays it at current prices
`GET /v1/validation`	→ predicted vs. reported GPU-hours per run and engine version

Technical Stack

Engine: Python 3.12, Pydantic v2, NumPy. **API and collectors:** FastAPI, SQLAlchemy, httpx, BeautifulSoup. **Database:** Postgres on Neon. **Frontend:** Next.js 15, TypeScript, Tailwind, Recharts, `@vercel/og`. **Testing and tooling:** pytest, Hypothesis, Playwright, uv, Ruff, mypy strict, pre-commit. **Deployment:** Vercel (web), Fly.io (API), GitHub Actions (CI, nightly cron, preview deploys).

Agent Readiness

- **Spec as source of truth.** `MODELING.md` holds every equation, unit and invariant, cited from each engine function; `CLAUDE.md` covers layout (`engine/`, `api/`, `collectors/`, `web/`), commands, and three rules: no I/O in `engine/`, no silent unit conversions, every PR adds a test.
- **Fast, reproducible checks.** Units in field names (`memory_bytes`), mypy strict and Ruff on pre-commit, `make check` under 60 s; `uv sync` plus Docker Compose with seeded prices and a `.devcontainer` matching CI.
- **Tests as the contract.** Hypothesis invariants and golden tests on published runs are the acceptance criteria; the AI PR reviewer follows `.github/review-checklist.md`; issue templates name the equation, expected result and failing test so a task can go

straight to an agent.

Communication & Presentation

Requirements reach humans and agents through the same artifacts: the spec files, issue templates, the project board, and `docs/prompts.md`, which logs each significant prompt and how it changed. The 5–8 minute demo-day talk follows one live scenario: context (1 min), a 70B training estimate with its share link and the validation page (3 min), the spec-driven agent workflow with before/after prompts (1–2 min), and results vs. the success metrics, where AI tools helped or failed, and the roadmap (1–2 min).